Structured Programming

COMS10015

Dr Tom Deakin

University of Bristol

Week 19

From computer processors to software applications

Programming computers

- So far focused on micro-architecture: the implementation of an ISA.
- Looked at optimising the micro-architecture with pipelining.
- Looked at the memory hierarchy, caches and how caches work.
- In the labs, building your own computer that implements the Hex 8 ISA.
- Next we shift focus and look at programming processors:
  - Writing programs in assembly and programming languages.
  - Software to turn those languages into machine code that can be executed by the processor.
  - How that software interacts with the hardware.

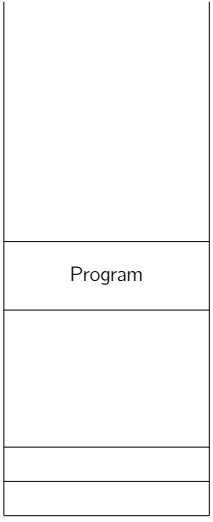
Processor Memory

Programs

- Programs are stored in memory.
- Programs consist of:
  - Instructions.
  - Data: global and local variables.
  - Large data structures: arrays and records.
  - A stack, to support subroutine calls.

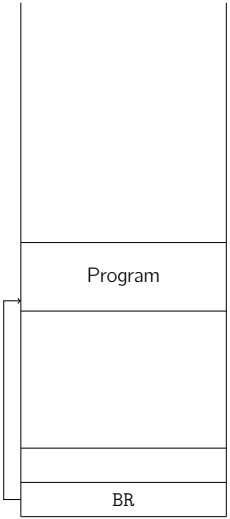
Processor memory (1)

- Program binary loaded into memory.



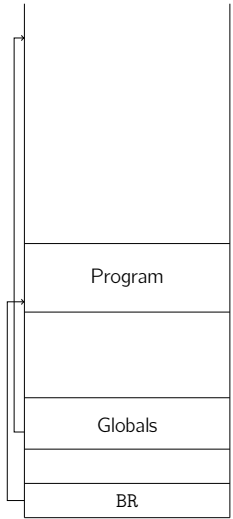
Processor memory (1)

- Program binary loaded into memory.
- When processor starts execution, *pc* is zero.
  - In *mem*[0] put jump instruction to start of program in memory.
  - First thing it executes is this branch to the start of the program.



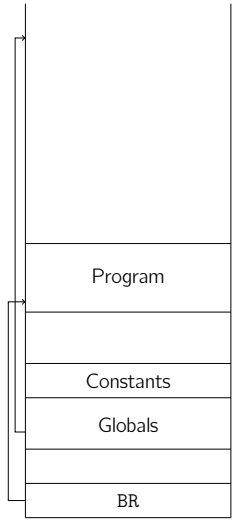
Processor memory (2)

- Between these are:
  - Global variables, which point to larger data structures.



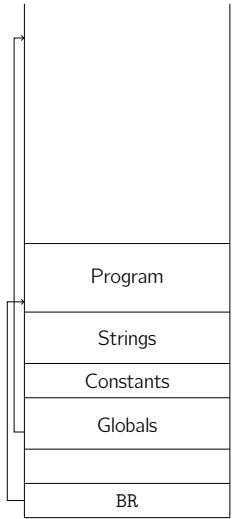
Processor memory (2)

- Between these are:
  - Global variables, which point to larger data structures.
  - Constant values defined in the program.



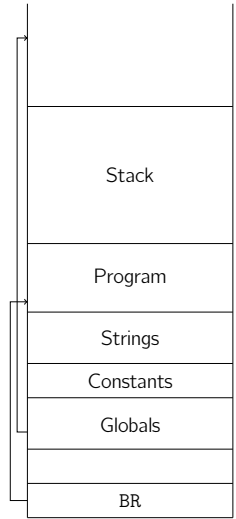
Processor memory (2)

- Between these are:
  - Global variables, which point to larger data structures.
  - Constant values defined in the program.
  - Strings defined in the program.



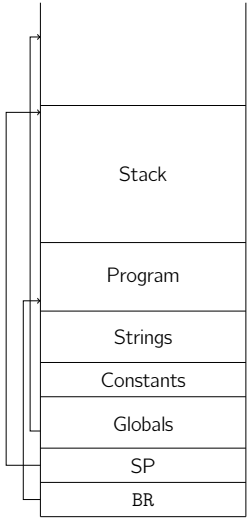
Processor memory (2)

- Between these are:
  - Global variables, which point to larger data structures.
  - Constant values defined in the program.
  - Strings defined in the program.
- The stack, used to store:
  - Local variables
  - Support call of subroutines.



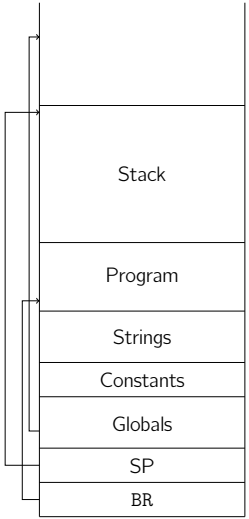
Processor memory (2)

- Between these are:
  - Global variables, which point to larger data structures.
  - Constant values defined in the program.
  - Strings defined in the program.
- The stack, used to store:
  - Local variables
  - Support call of subroutines.
- Stack pointer, which shows current location of top of stack.



Processor memory (2)

- Between these are:
  - Global variables, which point to larger data structures.
  - Constant values defined in the program.
  - Strings defined in the program.
- The stack, used to store:
  - Local variables
  - Support call of subroutines.
- Stack pointer, which shows current location of top of stack.
- Program binary has this structure, so memory set up when program loaded.



The Stack

- The stack provides storage for local variables.
  - These variables only exist for the duration of part of the program.
  - In C, this scope primarily determined by curly brackets: { }
- The stack is a Last-In-First-Out (LIFO) queue.
  - Typically a descending queue, where it grows downwards as data pushed on.
- Stacks keep track of state to support calling subroutines.

Stack overflow

- As the stack grows down in memory, it gets closer to the program.
- The stack has a limited size: the gap between the end of the program and the starting stack pointer.
- If the stack pointer exceeds this limit, we have a *stack overflow*.
- A type of *buffer overflow*, where data is written beyond the bounds and overwrites something it shouldn't.
- Here that's the program!
- Processors usually have some form of memory protection to report this error.

Subroutine Calls

Subroutines

Programs written in high level languages often have subroutines to break program up into specific tasks.
E.g. in C

```
void compute_pi(float *res)
{
  /* code */
}
```

Program counter (PC) will jump to instructions for subroutine, execute them, then jump back to continue after the jump.
Exactly how this is implemented is dependent on the ISA (and programming language): called the Application Binary Interface (ABI)

Subroutine calls using the stack

In calling a subroutine, the PC is changed to start executing at another location.
Number of steps:

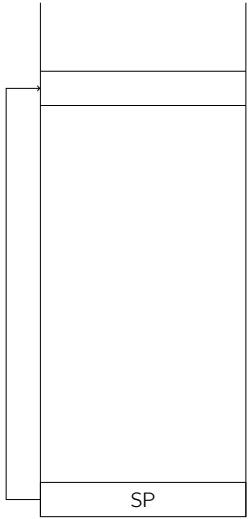
- Caller local variables and subroutine parameters pushed to stack (if required).
- Return address is pushed to stack.
- Stack pointer decreased by an offset to give space for local variables of *callee*.
- To return from the subroutine, stack pointer increased by offset.
  - This is the top of the original stack.
- Return address popped and used for jump instruction.
- Caller can pop parameters/local variables (if required).

Supporting subroutines in a program written in Hex 8

- Going to walk through using a stack for calling subroutines on a Hex 8 processor.
- A program binary will be loaded into the Hex 8 memory, and then the processor started.
- That binary will contain all parts from the previous section: jump, stack pointer, data, program, ...
- In reality, will use a *compiler* to generate the instruction sequence, rather than write them by hand.
- But will look at what that sequence is, and how it works.

Using a stack for subroutine calling in Hex 8 (1)

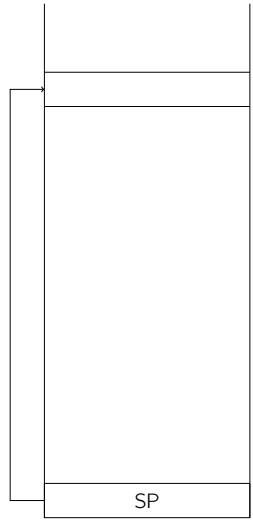
- When calling a routine, in Hex 8 the calling sequence would be:
  `0x10:` `LDAP 1`
  `0x11:` `BR FUNC`
  `0x12:` Resume execution here
- `LDAP` *areg* $\leftarrow$ *pc* + *oreg*
- This saves address of instruction to return to after the call in *areg* = `0x12`.
- `BR` updates PC to start of subroutine.



Using a stack for subroutine calling in Hex 8 (2)

```
0x10:  LDAP 1 (areg = PC+1 = 0x12)
0x11:  BR FUNC
0x12:  ...
...
0x40:  FUNC:
0x41:  LDBM 1 (breg = SP)
0x42:  STAI 0
```

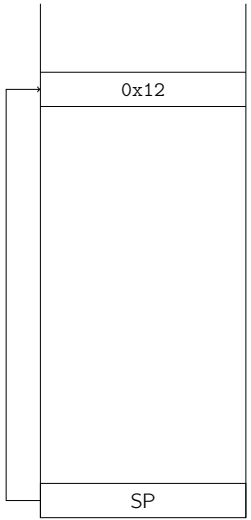
- At the start of subroutine, need to store the return address in the stack.
- `LDBM` looks up address of top stack, held in *breg*.



Using a stack for subroutine calling in Hex 8 (2)

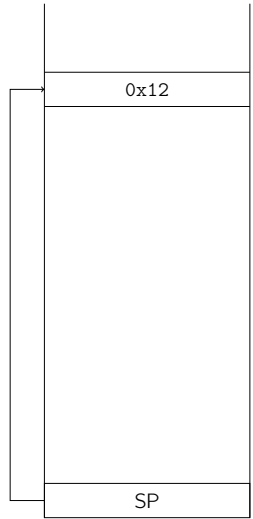
```
0x10:  LDAP 1 (areg = PC+1 = 0x12)
0x11:  BR FUNC
0x12:  ...
...
0x40:  FUNC:
0x41:  LDBM 1 (breg = SP)
0x42:  STAI 0
```

- At the start of subroutine, need to store the return address in the stack.
- `LDBM` looks up address of top stack, held in *breg*.
- `STAI` $mem[breg + oreg] \leftarrow areg$.



Using a stack for subroutine calling in Hex 8 (3)

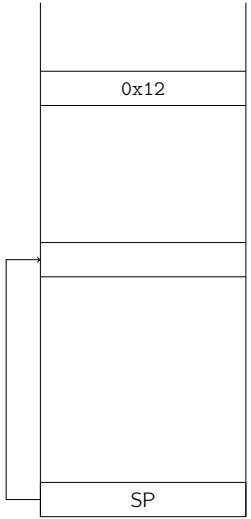
```
0x40:  FUNC:
0x41:  LDBM 1
0x42:  STAI 0
0x43:  LDAC -5 (areg = -5)
0x44:  ADD
(areg = areg+breg = -5 + old SP)
0x44:  STAM 1
```



Using a stack for subroutine calling in Hex 8 (3)

```
0x40:  FUNC:
0x41:  LDBM 1
0x42:  STAI 0
0x43:  LDAC -5 (areg = -5)
0x44:  ADD
(areg = areg+breg = -5 + old SP)
0x44:  STAM 1
```

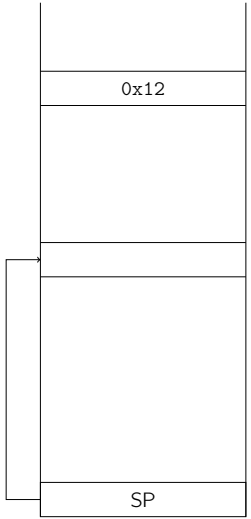
- SP then decremented to give enough space for local variables.



Using a stack for subroutine calling in Hex 8 (3)

```
0x40:  FUNC:
0x41:  LDBM 1
0x42:  STAI 0
0x43:  LDAC -5 (areg = -5)
0x44:  ADD
(areg = areg+breg = -5 + old SP)
0x44:  STAM 1
```

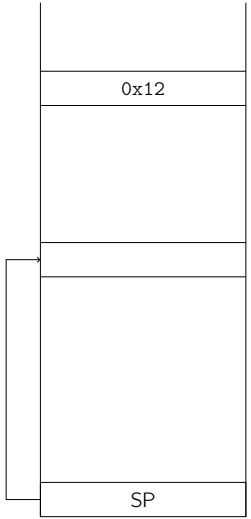
- SP then decremented to give enough space for local variables.
- Moving SP allows for another subroutine to be called.
- Subroutine code then follows.



Using a stack for subroutine calling in Hex 8 (4)

```
...
0x51:  LDBM 1 (breg = SP)
0x52:  LDAC 5 (areg = 5)
0x53:  ADD (areg=SP+5)
0x54:  STAM 1 (SP updated)
0x55:  LDBI 5
(breg = mem[old SP + 5] = ret
addr)
0x56:  BRB (pc = breg)
```

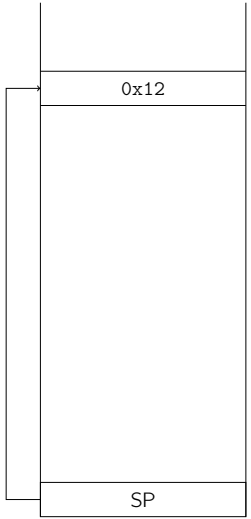
- To return from subroutine, load SP address, and add offset to get return address.



Using a stack for subroutine calling in Hex 8 (4)

```
...
0x51:  LDBM 1 (breg = SP)
0x52:  LDAC 5 (areg = 5)
0x53:  ADD (areg=SP+5)
0x54:  STAM 1 (SP updated)
0x55:  LDBI 5
(breg = mem[old SP + 5] = ret
addr)
0x56:  BRB (pc = breg)
```

- To return from subroutine, load SP address, and add offset to get return address.



Using a stack for subroutine calling in Hex 8 (5)

Caller

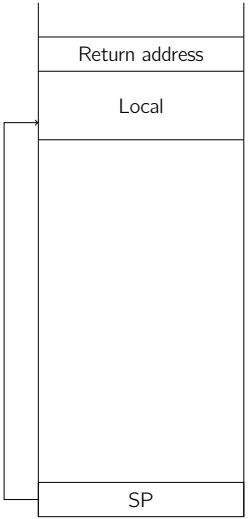
```
0x10: LDAP 1
0x11: BR 0x41
0x12: <return>
...
```

Callee

```
0x40: FUNC:
0x41: LDBM 1
0x42: STAI 0
0x43: LDAC -5
0x44: ADD
0x45: STAM 1
0x46: <FUNC body>
...
0x51: LDBM 1
0x52: LDAC 5
0x53: ADD
0x54: STAM 1
0x55: LDBI 5
0x56: BRB
```

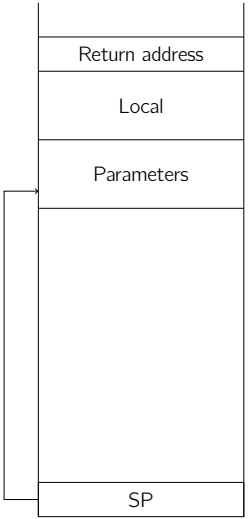
Local variables and parameters

- Each subroutine will have local variables on the stack, which exist only during scope of subroutine.



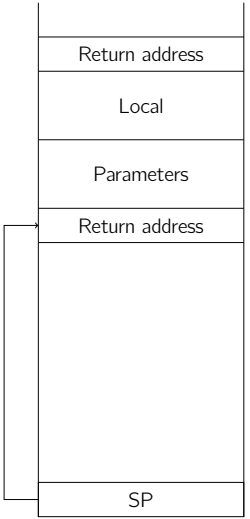
Local variables and parameters

- Each subroutine will have local variables on the stack, which exist only during scope of subroutine.
- Before calling another subroutine, the caller pushes parameters/arguments to the stack.



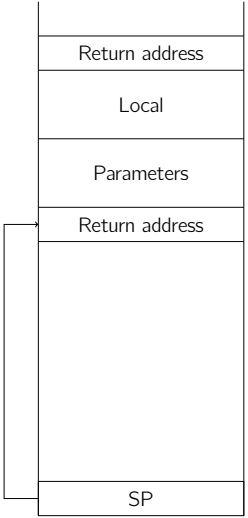
Local variables and parameters

- Each subroutine will have local variables on the stack, which exist only during scope of subroutine.
- Before calling another subroutine, the caller pushes parameters/arguments to the stack.
- Return address then pushed onto stack and branch executed.



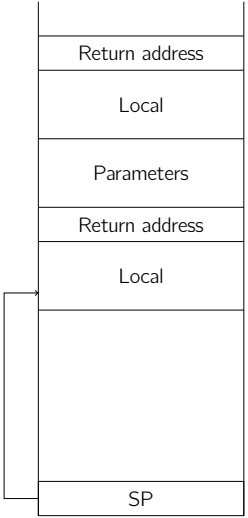
Local variables and parameters

- Each subroutine will have local variables on the stack, which exist only during scope of subroutine.
- Before calling another subroutine, the caller pushes parameters/arguments to the stack.
- Return address then pushed onto stack and branch executed.
- Callee can access the parameters: these are at `SP+1` on entry to subroutine.



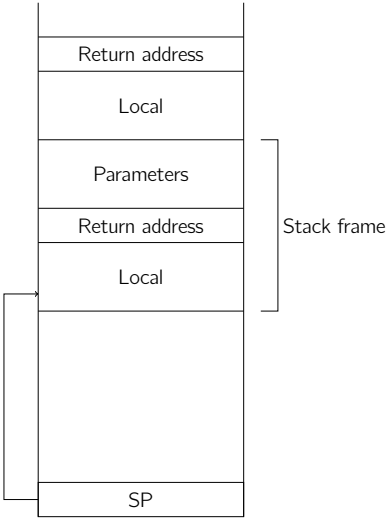
Local variables and parameters

- Each subroutine will have local variables on the stack, which exist only during scope of subroutine.
- Before calling another subroutine, the caller pushes parameters/arguments to the stack.
- Return address then pushed onto stack and branch executed.
- Callee can access the parameters: these are at `SP+1` on entry to subroutine.
- Callee has own local variables.



Local variables and parameters

- Each subroutine will have local variables on the stack, which exist only during scope of subroutine.
- Before calling another subroutine, the caller pushes parameters/arguments to the stack.
- Return address then pushed onto stack and branch executed.
- Callee can access the parameters: these are at `SP+1` on entry to subroutine.
- Callee has own local variables.
- This context is called the *stack frame*.



Stack Pointer Registers

- Processors often use a special register to keep stack pointer.
- Makes value directly accessible, don't need to first load from memory into general register.
- Example: Arm has Stack Pointer register, and Link register to hold return address of subroutines.
  - Recall Control flow lecture: branch-and-link
- Example: x86 has Stack pointers to top and bottom of stack frame (ESP and EBP respectively).
- ISA has rules for where these registers can be used.

Further Reading

- Stack: https://en.wikipedia.org/wiki/Stack_(abstract_data_type)
- Call stack: https://en.wikipedia.org/wiki/Call_stack
- Function calls: https://zhu45.org/posts/2017/Jul/30/understanding-how-function-call-works/
- Stack overflow: https://en.wikipedia.org/wiki/Stack_overflow
- https://en.wikipedia.org/wiki/Buffer_overflow
- https://inst.eecs.berkeley.edu/~cs161/fa08/papers/stack_smashing.pdf
- Subroutine calling in Intel processors: http://cse.unl.edu/~goddard/Courses/CSCE351/IntelArchitecture/IntelInterupts.pdf